



KE&M2: Practical Guide to Building OWL Ontology

Hutchatai Chanlekha

Department of Computer Engineering, Kasetsart University

Email: fenghtc@ku.ac.th





Reference

- Horridge, Matthew. (2011). A Practical Guide To Building OWL Ontologies Using Protégé 4 and CO-ODE Tools Edition 1.3.
 - [https://www.researchgate.net/publication/272829948 A Practical Guide To Building OWL Ontologies Using Protege 4 and CO-ODE Tools Edition 13](https://www.researchgate.net/publication/272829948_A_Practical_Guide_To_Building_OWL_Ontologies_Using_Protege_4_and_CO-ODE_Tools_Edition_13)



Protégé Reasoner for today's class

- In your Protégé
 - Check the menu Reasoner to see whether you have "Pellet"
 - If not, go to menu File > Check for Plugins, look for "Pellet"
 - Choose "Pellet" and click install



ONTOLOGY REVIEW



Component of OWL Ontology

- Individuals
- Properties
- Class



Component of OWL Ontology: Individuals

- Individuals, represent objects in the domain in which we are interested
- OWL **does NOT use the Unique Name Assumption (UNA)**. This means that two different names could actually refer to the same individual
- In OWL, it must be explicitly stated that individuals are the same as each other, or different to each other
 - Provides explicit constructs to express whether two names denote the same or distinct entities
 - `owl:sameAs`
 - `owl:differentFrom`

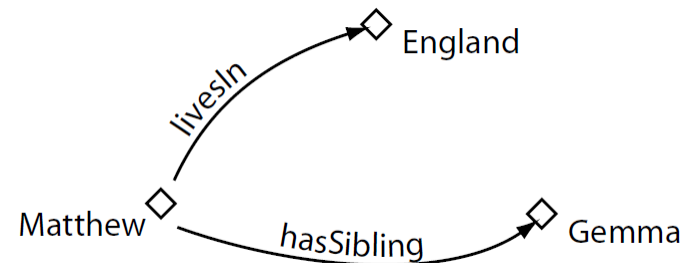


Individuals are also known as instances. Individuals can be referred to as being 'instances of classes'.



Component of OWL Ontology: Properties

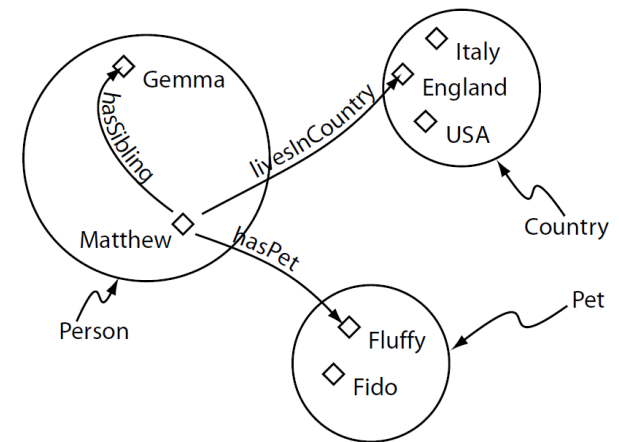
- Properties (Object properties) are binary relations on individuals - i.e. properties link two individuals together
 - Example: property `hasSibling` link the individual `Matthew` to the individual `Gemma`
- Properties can have characteristics
 - Example: Inverse, transitive, symmetric



Properties are also known as roles in description logics and relations in UML and other object-oriented notions. In some other formalisms they are called attributes.

Component of OWL Ontology: Classes

- OWL classes are interpreted as sets that contain individuals.
- They are described using formal descriptions that state precisely the requirements for membership of the class.
- In OWL, classes are built up of descriptions that specify the conditions that must be satisfied by an individual for it to be a member of the class.



The word “concept” is sometimes used in place of class. Classes are a concrete representation of concepts.

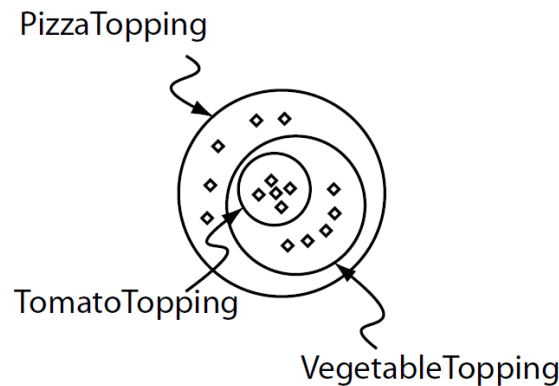


Classe - subclass

- Classes may be organized into a superclass-subclass hierarchy, which is also known as a taxonomy.
 - Subclasses specialize (i.e. are subsumed by) their superclasses
- For example: class `Animal` and `Cat`
 - `Cat` might be a subclass of `Animal` (so `Animal` is the superclass of `Cat`).
 - We can say that: *“All cats are animals”, “All members of the class `Cat` are members of the class `Animal`”, “Being a `Cat` implies that you're an `Animal`”, and “`Cat` is subsumed by `Animal`”.*
- One of the key features of OWL-DL is that these superclass-subclass relationships (i.e. subsumption relationships) can be computed automatically by a reasoner

Class-Subclass Relationship

- In OWL subclass means necessary implication.
- In other words, if A is a subclass of B, then all instances of A are instances of B without exception
- For example:



- If VegetableTopping is a subclass of PizzaTopping then ALL instances of VegetableTopping are instances of PizzaTopping.
- If something is a VegetableTopping then this implies that it is also a PizzaTopping



Let's dig deep into 'CLASS'



Let's look at the Pizza ontology

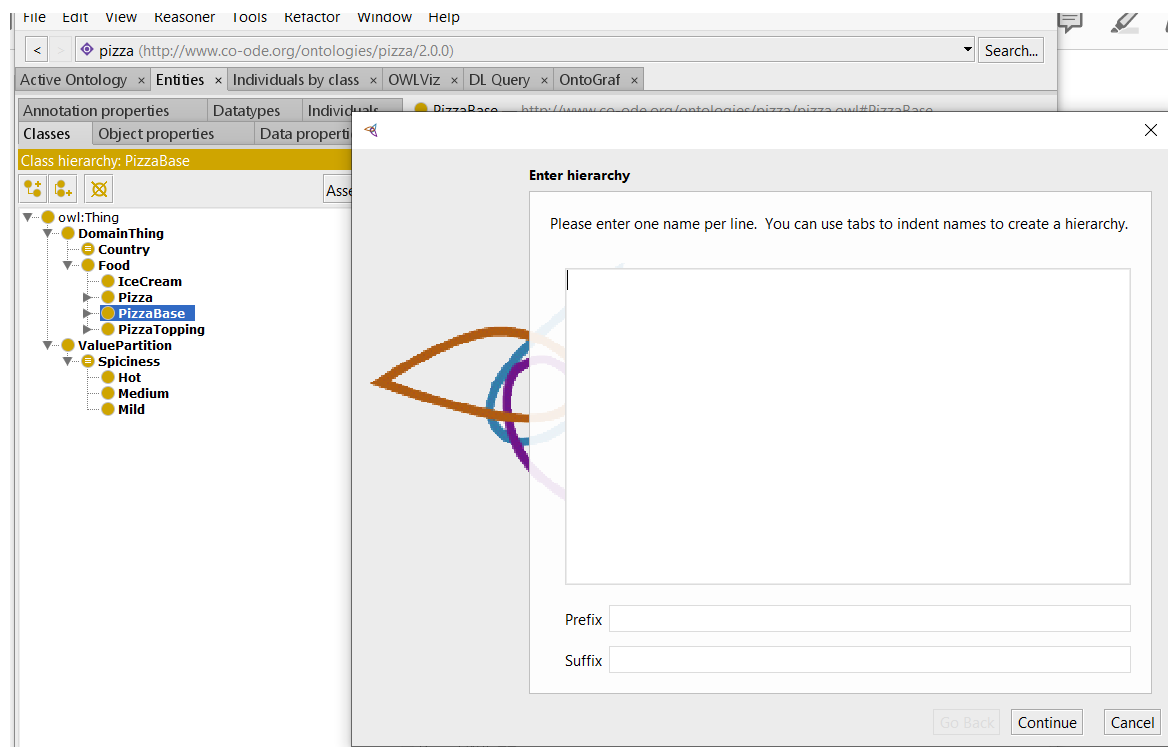
- Start the Protégé
- Open the file pizza.owl
- Edit the ending of default URI with <http://.../<id>-pizza-ontology>
- Save as to a new OWL/XML-format file

Add a comment to ontology

1. Select “Active Ontology Tab”
2. In the “Annotations” view, click the “Add” icon (+), An editing window will appear in the table. Select “comment” from the list of built in annotation URIs and type your comment in the text box
3. Enter a comment such as “[A pizza ontology that describes various pizzas based on their toppings.](#)” and press OK to assign the comment.

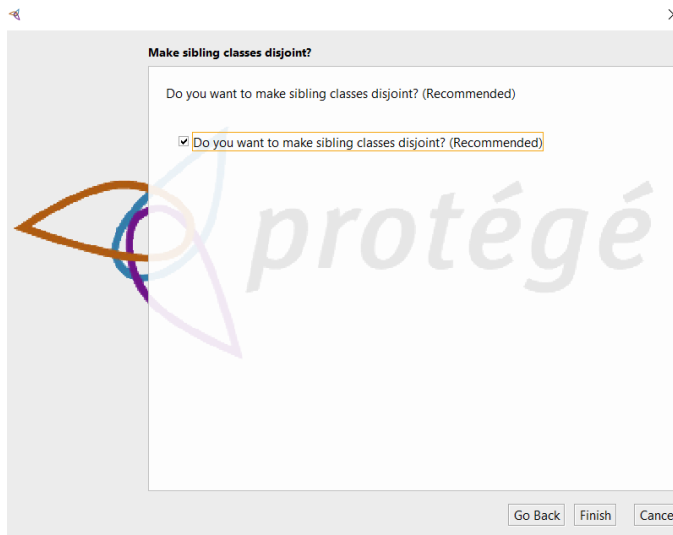
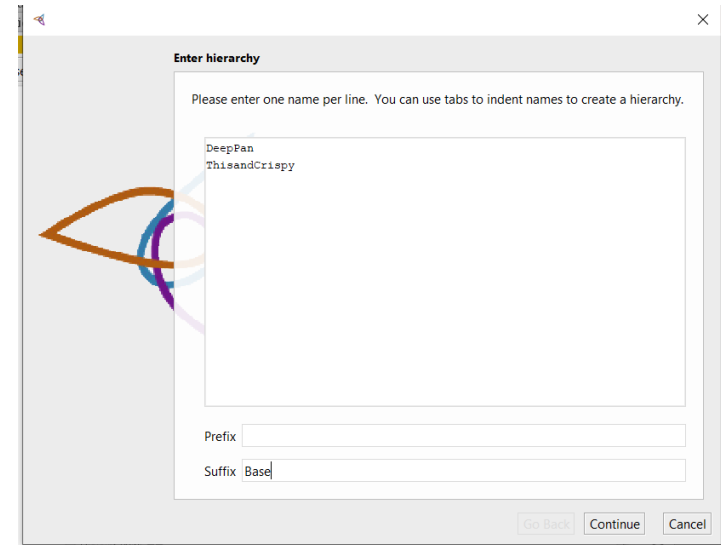
“Create Class Hierarchy” tool in Protégé

- We can use “Create Class Hierarchy” tool to create subclass
 - Tools menu > Create Class Hierarchy



“Create Class Hierarchy” tool in Protégé

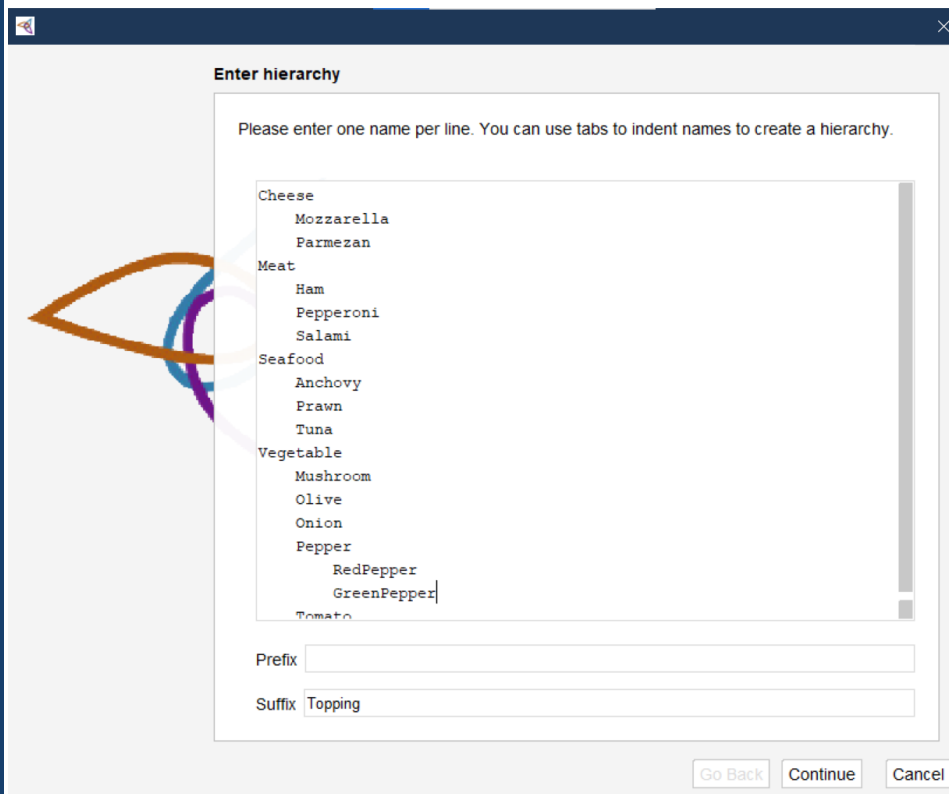
- We can set the value of prefix/suffix Instead of typing directly for every class



- By checking the box “*Make sibling classes disjoint*”, the tool will automatically make the new classes disjoint

“Create Class Hierarchy” tool in Protégé

- Suppose we create multiple classes in a hierarchy by using Create Class Hierarchy tool under Topping class
- After click finish, we will class hierarchy



Enter hierarchy

Please enter one name per line. You can use tabs to indent names to create a hierarchy.

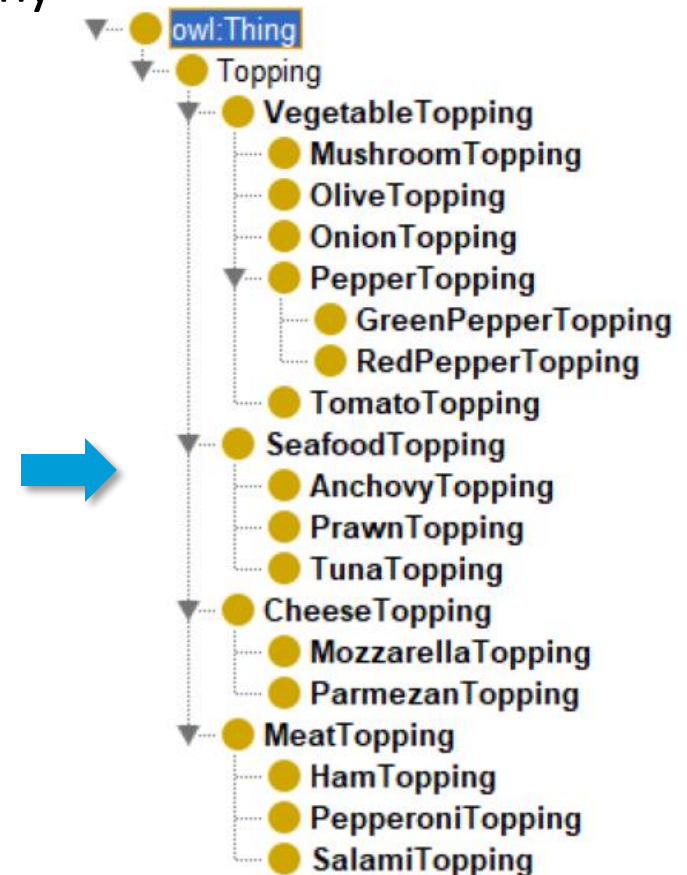
```

Cheese
  Mozzarella
  Parmezan
Meat
  Ham
  Pepperoni
  Salami
Seafood
  Anchovy
  Prawn
  Tuna
Vegetable
  Mushroom
  Olive
  Onion
  Pepper
    RedPepper
    GreenPepper
  Tomato
  
```

Prefix

Suffix

Go Back Continue Cancel





Disjoint Class

- “Disjoint classes”
 - An individual cannot be an instance of more than one of disjointed classes.
- If we want to state that two classes are disjoint,
 - Click on “Entity” tab
 - Click on the class that you want to make disjoint
 - On the class “Description” pane, click “Add” icon (+) next to “Disjoint with”, and choose the classes that are disjoint with the selected class



Let's look at Pizza ontology

The screenshot shows the Protege ontology editor interface. The title bar indicates the file is 'pizza (http://www.co-ode.org/ontologies/pizza/2.0.0)'. The menu bar includes File, Edit, View, Reasoner, Tools, Refactor, Window, and Help. The toolbar contains icons for navigating between different views: Active Ontology, Entities, Individuals by class, OWLViz, DL Query, and OntoGraf. The main workspace is divided into several panes. On the left, the 'Class hierarchy: Pizza' pane shows a tree structure starting from 'owl:Thing', with 'DomainThing' as a subclass. 'DomainThing' has a subclass 'Country', which has a subclass 'Food'. 'Food' has subclasses 'IceCream' and 'Pizza'. 'IceCream' has a subclass 'PizzaBase', which in turn has a subclass 'PizzaTopping'. 'Pizza' has a subclass 'ValuePartition', which has a subclass 'Spiciness'. 'Spiciness' has subclasses 'Hot', 'Medium', and 'Mild'. The 'Annotations: Pizza' pane on the right shows three annotations: 'rdfs:label [language: en] Pizza', 'skos:prefLabel [language: en] Pizza', and 'rdfs:seeAlso https://en.wikipedia.org/wiki/Pizza'. The 'Description: Pizza' pane at the bottom right shows the following axioms: 'Equivalent To' (empty), 'SubClass Of' (Food, hasBase PizzaBase), 'General class axioms' (empty), 'SubClass Of (Anonymous Ancestor)' (empty), 'Instances' (empty), 'Target for Key' (empty), 'Disjoint With' (IceCream, PizzaTopping, PizzaBase), and 'Disjoint Union Of' (empty).

Try adding Pasta class as a subclass of Food and create disjoint class for Pasta



Disjoint Class - Remark

- OWL Classes **are assumed to `overlap`**.
- In order to `separate' a group of classes, we must make them disjoint from one another.
- This ensures that an individual which has been asserted to be a member of one of the classes in the group cannot be a member of any other classes in that group.

When two classes are disjoint, this means that it is not possible for an individual to be a member of a combination of these classes

Notice that both subclasses of PizzaBase in Pizza ontology; “DeepPanBase” and “ThinandCrispyBase” are disjoint.

- Because a pizza base cannot be both thin and crispy and deep pan



Enumerated Class

- OWL allows classes to be defined by precisely listing the individuals that are the members of the class.
- For example, we might define a class *DaysOfTheWeek* to contain only the individuals:
{Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday}
- Classes such as this are known as **enumerated classes**

To create enumerated classes

- First, the individuals must have been created in the ontology
- Then select the class you want to make enumerated class
- On Description pane, click add (+) next to the “Equivalent To”
- Click on tab “Class Expression Editor”
- Type individual names you want to define as the only member of the class
 - {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday}



Review Question

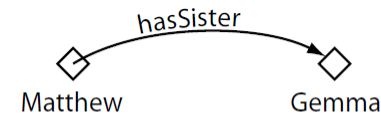
- What is the definition of disjoint class
- What is enumerated class???



Let's dig deep into 'Property'

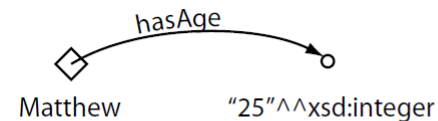
OWL Properties

- OWL Properties represent relationships.
- Two main types of properties
 - Object properties



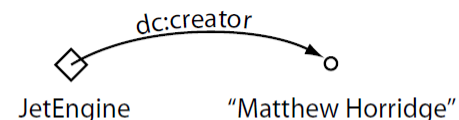
An object property linking the individual Matthew to the individual Gemma

- Datatype properties



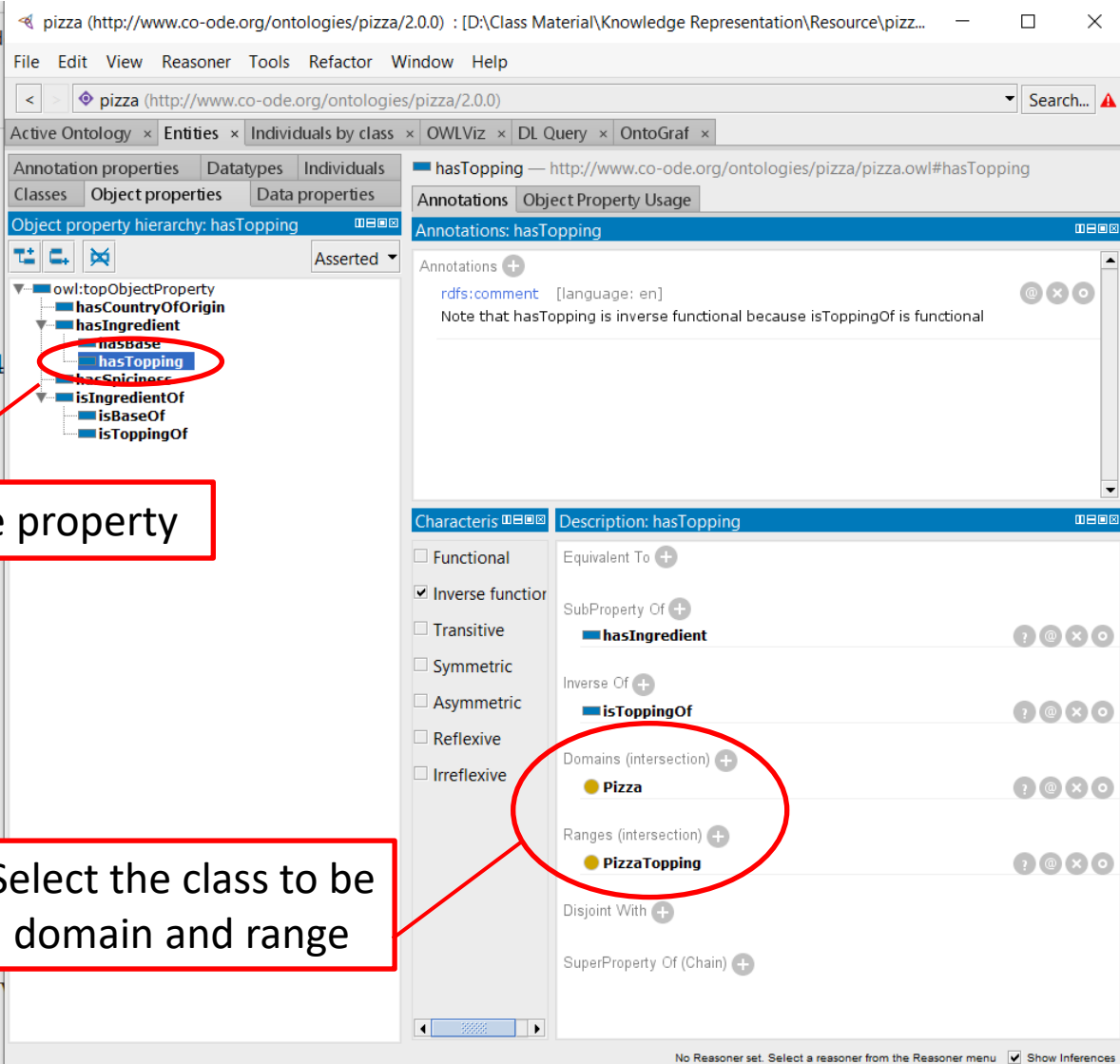
A datatype property linking the individual Matthew to the data literal '25', which has a type of an xsd:integer.

- OWL also has a third type of property, Annotation properties
 - Annotation properties can be used to add information (metadata) to classes, individuals and object/datatype properties



An annotation property, linking the class 'JetEngine' to the data literal (string) "Matthew Horridge".

To set domain and range in Protégé



Active Ontology: pizza (http://www.co-ode.org/ontologies/pizza/2.0.0)

Object property hierarchy: hasTopping

Object properties: hasCountryOfOrigin, hasIngredient, hasBase, hasTopping, hasSpiciness, isIngredientOf, isBaseOf, isToppingOf

Annotations: hasTopping

Annotations: rdfs:comment [language: en]
Note that hasTopping is inverse functional because isToppingOf is functional

Characteristics: Functional, Inverse function, Transitive, Symmetric, Asymmetric, Reflexive, Irreflexive

Description: hasTopping

Equivalent To: +

SubProperty Of: + hasIngredient

Inverse Of: + isToppingOf

Domains (intersection): + Pizza

Ranges (intersection): + PizzaTopping

Disjoint With: +

SuperProperty Of (Chain): +

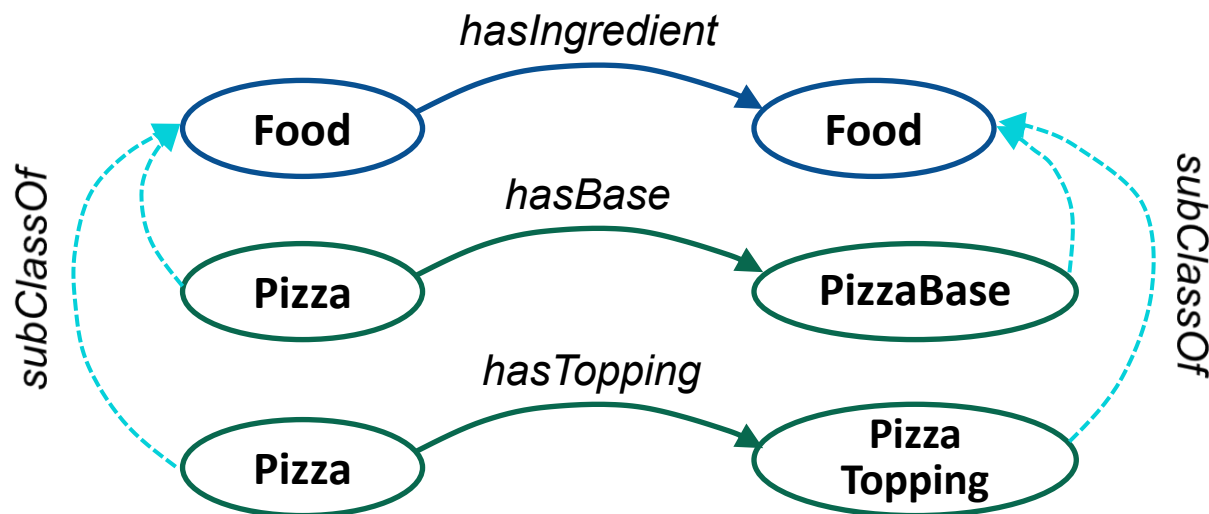
Select the property

Select the class to be domain and range



Property – Sub property

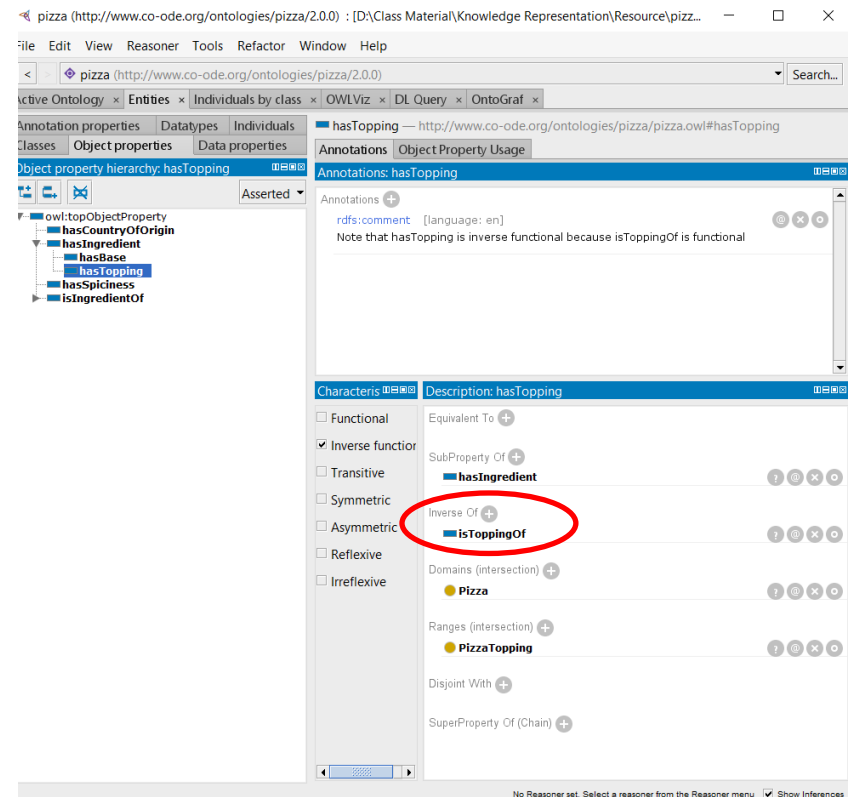
- In OWL, properties may have sub properties
 - We can form hierarchies of properties.
- Sub-properties specialize their super properties (in the same way that subclasses specialize their super-classes)
- Example: In the case of our pizza ontology
 - Properties *hasTopping* and *hasBase* is sub-properties of *hasIngredient*





Inverse Properties

- Each object property may have a corresponding inverse property
- If some property links individual **a** to individual **b** then its inverse property will link individual **b** to individual **a**
- To specify inverse property





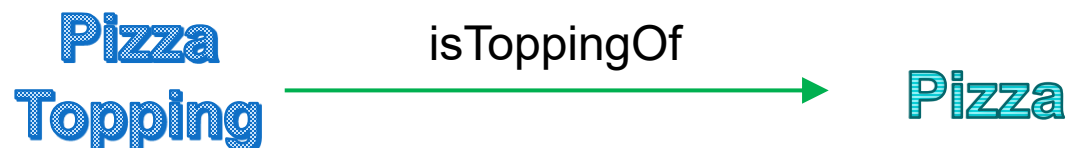
Domain and Range of Inverse property

- Once you set domain and range of a property **P**
- If **P** has inverse property **Q**, then Protégé will automatically identify domain and range of the **Q**
- Example:
 - isToppingOf property, which is the inverse property of hasTopping



Define **isToppingOf** as inverse of **hasTopping**

Protégé will automatically add PizzaTopping as domain and Pizza of Range for isToppingOf





Property Characteristics

- OWL allows the meaning of properties to be enriched through the use of property characteristics
- Characteristics that properties may have:
 - Functional properties
 - Inverse Functional properties
 - Transitive properties
 - Symmetric properties
 - Asymmetric properties
 - Reflexive properties
 - Irreflexive properties



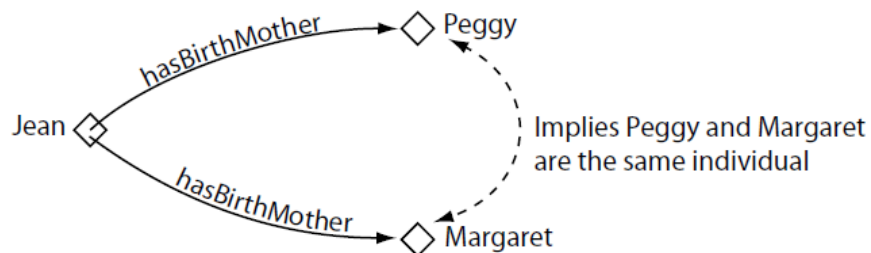
Functional Property

- If a property is functional, for a given individual, there can be at most one individual that is related to the individual via the property.
 - For example: functional property *hasBirthMother* -- something can only have one birth mother.
- Functional properties are also known *as single valued properties*

Suppose *hasBirthMother* is Functional Property

If you say “Jean *hasBirthMother* Peggy” and “Jean *hasBirthMother* Margaret”

Then Protege will infer that “*Peggy*” and “*Margaret*” are the same individual





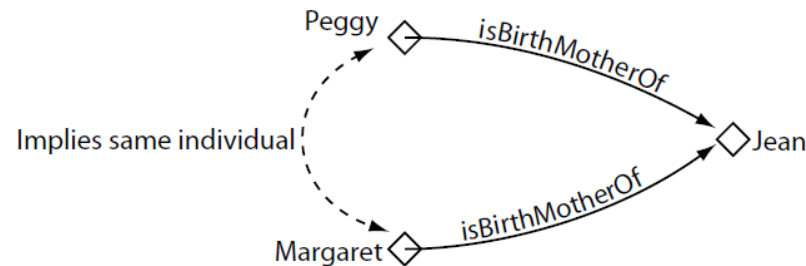
Let's try

- Create three individuals
 - `orderpizza` as an instance of *Pizza*
 - `crispybase` as an instance of *ThinAndCrispyBase*
 - `crunchybase` as an instance of *ThinAndCrispyBase*
- Check whether `hasBase` has functional characteristics
 - If not, make it functional
- Add property `hasBase` to *orderpizza*
 - `orderpizza` `hasBase` *crispybase*
 - `orderpizza` `hasBase` *crunchybase*
- Start reasoner
- Click on the instance `crispybase` and `crunchybase`, then see what happen



Inverse Functional Property

- If a property is inverse functional, then for any given value in its range, there can be at most one individual in the domain that is related to it.
 - For example: *isBirthMotherOf* is inverse functional property
 - *isBirthMotherOf* is the inverse property of *hasBirthMother*
 - Since *hasBirthMother* is functional, *isBirthMotherOf* is inverse functional.



If we state that Peggy is the birth mother of Jean,
and we also state that Margaret is the birth mother of Jean,
With inverse functional property,
we can infer that Peggy and Margaret are the same individual.

- If a property is inverse functional then it means its inverse property is functional.



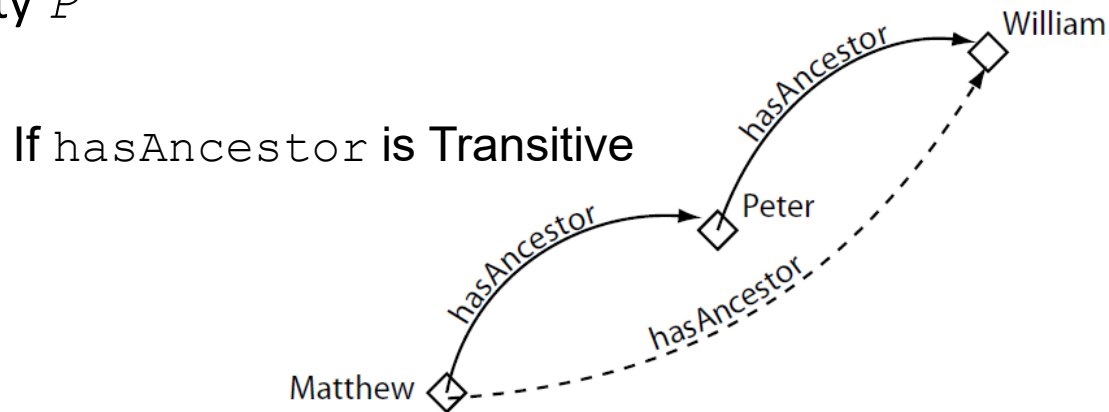
Let's look at the Pizza ontology

- What properties should be functional property?
 - hasIngredient, hasBase, hasTopping, hasSpiciness, isIngredientOf, isBaseOf, isToppingOf
- And what about inverse functional property?



Transitive Property

- A property P is transitive,
 - If the property P relates individual a to individual b , and also individual b to individual c
 - Then we can infer that individual a is related to individual c via property P



If a property is transitive, will its inverse property be transitive?

if a property is transitive, can be functional?



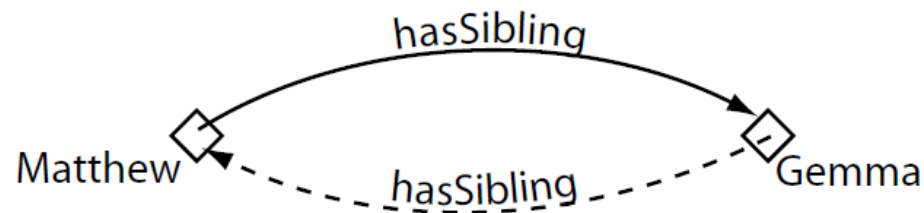
Let's look at the Pizza ontology

- What properties should have transitive property?
 - hasIngredient, hasBase, hasTopping, hasSpiciness, isIngredientOf, isBaseOf, isToppingOf



Symmetric Property

- A property P is Symmetric,
 - If the property P relates individual a to individual b ,
 - Then we can infer that individual b is also related to individual a via property P



If `hasSibling` is symmetric
and *Matthew* has a sibling that is *Gemma*
Then we can infer that *Gemma* must have a sibling that is *Matthew*



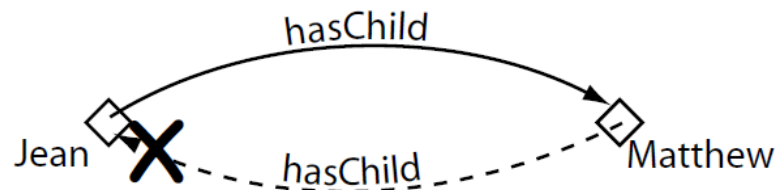
Question

- Can data property have the following characteristics?
 - Functional
 - Inverse functional
 - Transitive
 - Symmetric



Asymmetric Property

- If a property ***P*** is asymmetric, and the property relates individual *a* to individual *b*
 - Then individual *b* cannot be related to individual *a* via property ***P***

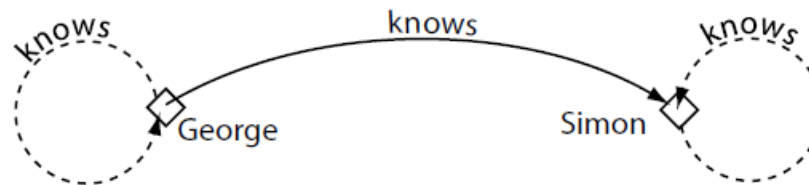


If hasChild is asymmetric and Robert is a child of David,
Then David cannot be a child of Robert



Reflexive Property

- A property ***P*** is said to be reflexive when the property must relate individual *a* to itself.
 - For example, everybody must know themselves besides knowing other people, so the property `Knows` should be reflexive property

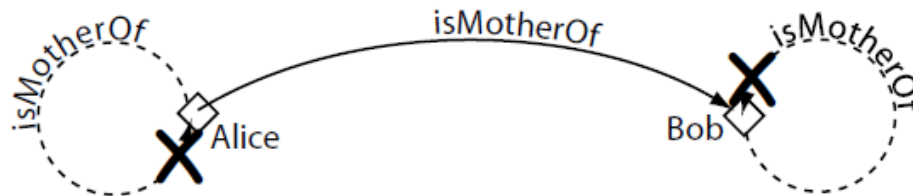


George must know himself;
and it is possible for George to know other people

Let's add object property "`sameTaste`" with *Reflexive* characteristics and start the Reasoner

Irreflexive Property

- A property ***P*** is irreflexive if it is a property that relates an individual *a* to individual *b*, where individual *a* and individual *b* cannot be the same.



No one can be mother of oneself



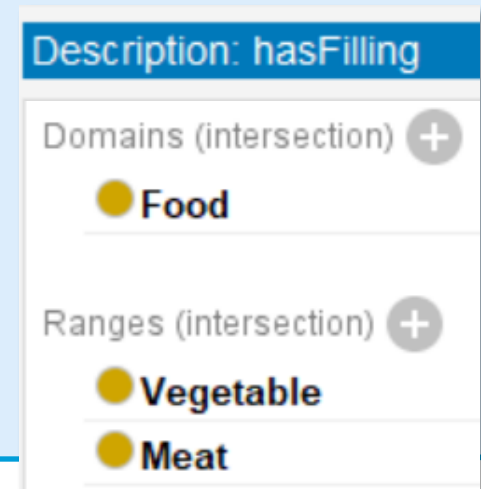
Property: Domain and Range

- Properties link individuals from the domain to individuals from the range
- Example:
 - Property hasTopping link individuals belonging to the class Pizza to individuals belonging to the class of PizzaTopping



Property: Domain and Range

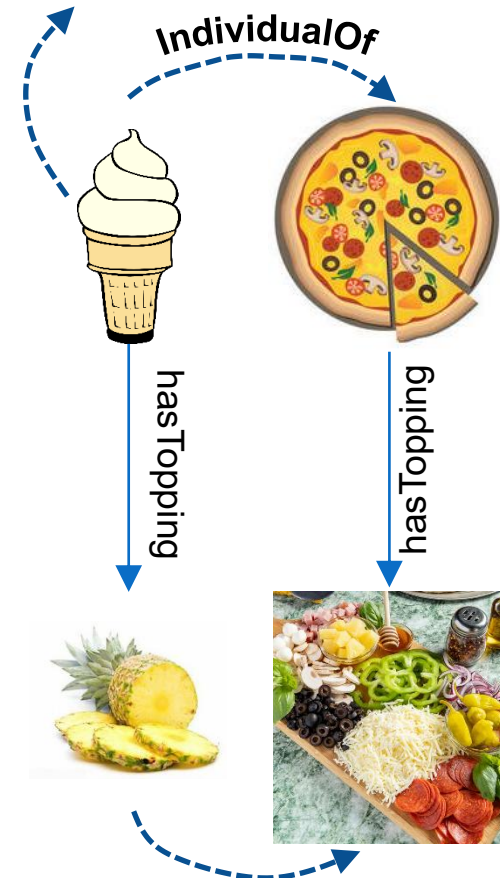
- We can specify multiple classes as the range for a property
- For Protégé 4 and 5
 - If multiple classes are specified in Protégé, the range of the property is interpreted to be the intersection of the classes.
- Example:
 - If the range of a property has the classes *Meat* and *Vegetable* listed in the range view,
 - The range of the property will be interpreted as *Meat* intersection *Vegetable*



What will happen if Meat and Vegetable are disjoint?

Property Domain and Range

- Note that in OWL domains and ranges **should not be viewed as constraints** to be checked.
- They are used as “**axioms**” in reasoning.
- Example: if the property *hasTopping* has the domain set as *Pizza*, range as *Topping*
 - But we applied the *hasTopping* property to Icecream1 (individual under the class IceCream)
 - Generally, it's not result in an error
 - It would be used to infer that the Icecream1 (the individual of class IceCream) is also an individual of class pizza
 - An error will only be generated (by a reasoner) if class Pizza is disjoint to class IceCream!



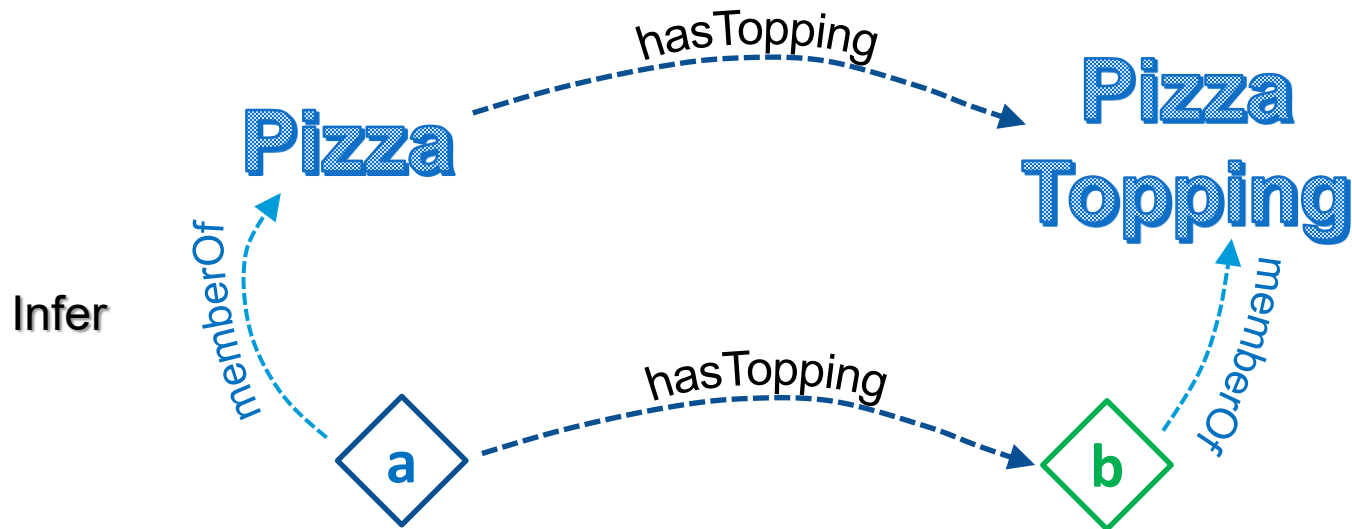
Error if class Pizza is set to be disjointed with class IceCream



Let's try

- Create “chocolateSunday” as individual of class IceCream
 - Create “almond” as individual of class NutTopping
 - Create property “hasTopping” for chocolateSunday
 - chocolateSunday *hasTopping* almond
 - Then start Reasoner
-
- Let's try removing “Pizza” from the disjoint class of “IceCream”
 - Then start Reasoner

Inference of Domain and Range



- If we have individuals *a* and *b* and an assertion that *a hasTopping b*
- Then it will be inferred that *a* is a member of the class *Pizza* and that *b* is a member of the class *PizzaTopping*
 - This will be the case even if *a* has not been asserted to be a member of the class *Pizza* and/or *b* has not been asserted to be a member of the class *PizzaTopping*



Let's try

- Create two individuals without stating their class
 - `Dish1`, `Ingredient1`
- **Add** `hasTopping` **property** to relate from `Dish1` to `Ingredient1`
- Run Reasoner
- What happen?



Difficulties that could arise from setting domain and range

- A point to be concerned because of these behaviors
 - The fact that domain and range conditions do not behave as constraints
 - And the fact that they can cause “unexpected” classification results
- They can lead problems and unexpected side effects.
- These problems and side effects can be particularly difficult to track down in a large ontology.

Special care has to be taken when specifying domain and range.

Some experts even generally advise against doing this.



Define class with property restrictions



Defining class with property restrictions

- A class can be described or defined by the relationships that its individual members participate in
- In OWL we can define such classes by using **restrictions**
 - A restriction describes a class of individuals based on the relationships that members of the class participate in



Restriction Examples

- The class of individuals that have at least one `hasTopping` relationship
- The class of individuals that have at least one `hasTopping` relationship to members of `VegetableTopping` – i.e. things that have at least one topping that is vegetable
- The class of individuals that have more than three `hasTopping` relationships
- The class of individuals that have at least one `hasTopping` relationship to individuals that are members of `MozzarellaTopping` – i.e. the class of things that have at least one kind of mozzarella topping
- The class of individuals that only have `hasTopping` relationships to members of `VegetableTopping` – i.e. the class of individuals that only have toppings that are vegetable toppings



Categories of OWL restrictions

- Restrictions in OWL fall into three main categories
 - Quantifier Restrictions
 - Existential restrictions
 - Universal restrictions
 - Cardinality Restrictions
 - hasValue Restrictions



Quantifier Restrictions

- Existential restrictions [some]
 - Describe classes of individuals that participate in **at least one relationship** along a specified property to individuals that are members of a specified class
 - For example: the class of individuals that have at least one (some) `hasTopping` relationship to members of `MozzarellaTopping`
 - Also known as “`someValuesFrom`” restrictions
- Universal restrictions [only]
 - Describe classes of individuals that for a given property, they only have relationships along this property to individuals that are members of a specified class
 - For example: the class of individuals that only have `hasTopping` relationships to members of “`VegetableTopping`”
 - Also known as “`allValuesFrom`” restrictions

Setting class restrictions

pizza (http://www.co-ode.org/ontologies/pizza/2.0.0) : [D:\Class Material\Knowledge Representation\Resource\pizz...

File Edit View Reasoner Tools Refactor Window Help

< > pizza (http://www.co-ode.org/ontologies/pizza/2.0.0) Search...

Active Ontology x Entities x Individuals by class x OWLViz x DL Query x OntoGraf x

Annotation properties Datatypes Individuals

Classes Object properties Data properties

Class hierarchy: Margherita

Annotations: Margherita

Annotations +

rdfs:label [language: en]
Margherita

rdfs:label [language: pt]
Margherita

skos:prefLabel [language: en]
Margherita

skos:altLabel [language: en]
Margherita Pizza

Description: Margherita

Equivalent To +

SubClass Of +

hasTopping **only**
(MozzarellaTopping or TomatoTopping)

hasTopping **some** MozzarellaTopping

hasTopping **some** TomatoTopping

NamedPizza

General class axioms +

SubClass Of (Anonymous Ancestor)

hasBase **some** PizzaBase

Instances +

Target for Key +

Disjoint With +

AmericanHot Fiorentina Cajun LaReine Giardiniera Rosa

Class hierarchy: Margherita

IceCream

Pizza

CheesyPizza

InterestingPizza

MeatyPizza

NamedPizza

American

AmericanHot

Cajun

Capricciosa

Caprina

Fiorentina

FourSeasons

FruttiDiMare

Giardiniera

LaReine

Margherita

Mushroom

Napoletana

Parmense

PolloAdAstra

PrinceCarlo

QuattroFormaggi

Rosa

Siciliana

SloppyGiuseppe

Soho

Veneziana

NonVegetarianPizza

RealItalianPizza

SpicyPizza

SpicyPizzaEquivalent

ThinAndCrispyPizza

UnclosedPizza

VegetarianPizza

VegetarianPizza1

VegetarianPizza2

PizzaBase

PizzaTopping

CheeseTopping

FruitTopping

HerbSpiceTopping

Create class restrictions

- Existential restrictions: some
- Universal restrictions: only



Existential Restrictions

- An existential restriction describes a class of individuals that have at least one (some) relationship along a specified property to an individual that is a member of a specified class.
- For example: `hasBase some PizzaBase`
 - Describes all of the individuals that have at least one relationship along the `hasBase` property to an individual that is a member of the class `PizzaBase`
- Existential restrictions are also known as *Some Restrictions*, or as *some values from* restrictions



Create Existential Restriction

- Suppose we want to create existential restriction to class CheesyPizza
- Click ⊕ behind 'Subclass of'
 - Add hasTopping some CheeseTopping

Description: CheesyPizza

Equivalent To ⊕

SubClass Of ⊕

- hasTopping some CheeseTopping
- Pizza

General class axioms ⊕

SubClass Of (Anonymous Ancestor)

- hasBase some PizzaBase

1. Click (+)
2. Select Tab "*Object Restriction Creator*"
3. Select Property
4. Select restriction type as "*Some*"
5. Then choose the class for restriction filler



Universal Restriction

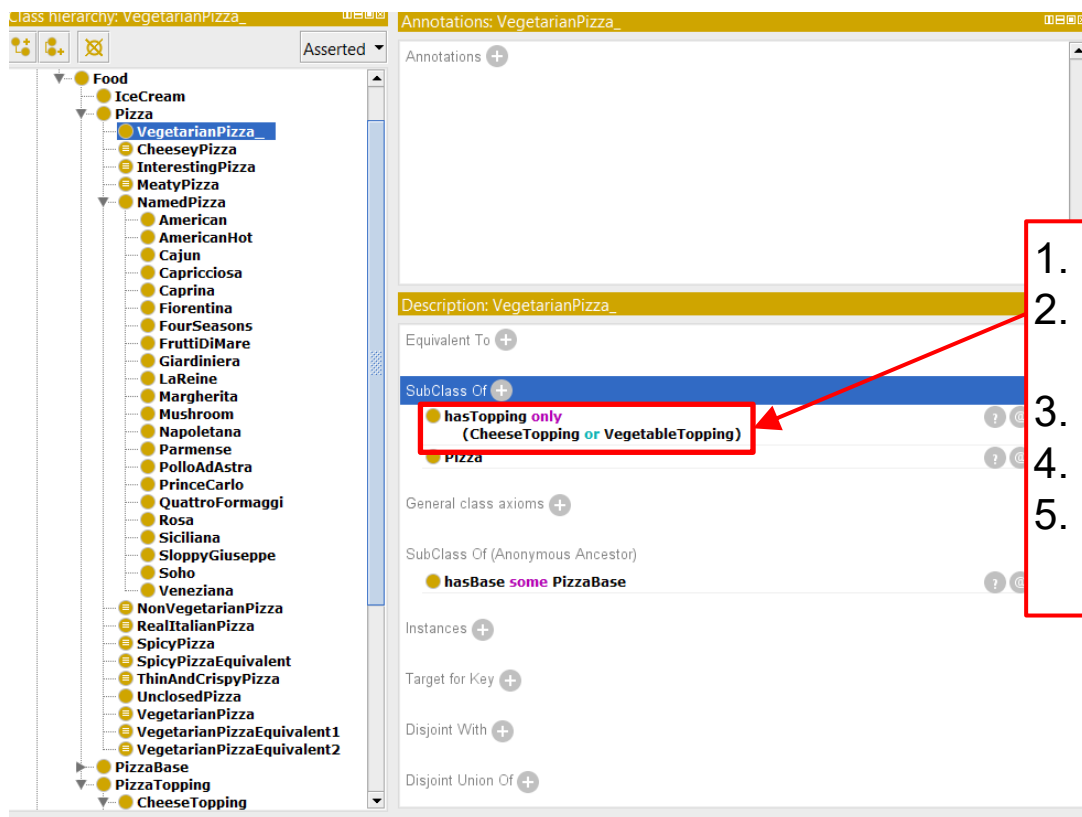
- Universal restrictions constrain the relationships along a given property to individuals that are members of a specific class.
- For example: ***hasTopping only MozzarellaTopping***
 - Describes the individuals all of whose *hasTopping* relationships are to members of the class *MozzarellaTopping*
 - Or you can say that the individuals do not have a *hasTopping* relationships to individuals that aren't members of the class *MozzarellaTopping*
- Remark:: the universal restriction ***hasTopping only MozzarellaTopping*** also describes the individuals that **do not participate in any *hasTopping* relationships**

Universal restrictions are also known as AllValuesFrom Restrictions



Create Universal Restriction

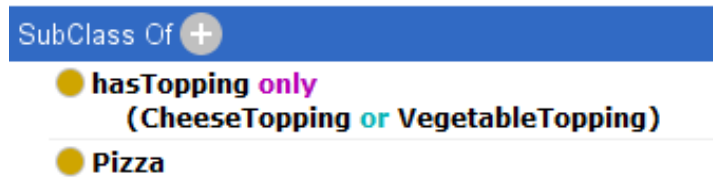
- Suppose we want to create a class called VegetarianPizza.
 - Individuals that are members of this class can only have toppings that are CheeseTopping or VegetableTopping.
- To do this we can use a *universal restriction*



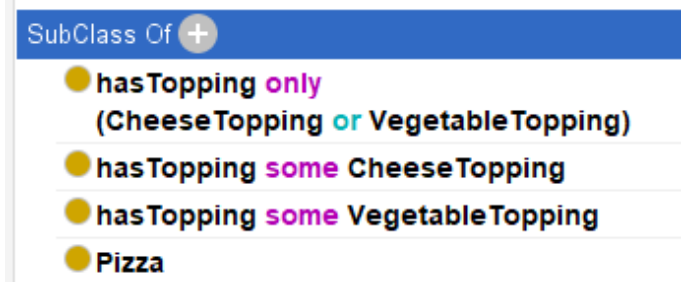
1. Click (+)
2. Select Tab “Object Restriction Creator”
3. Select Property
4. Select restriction type as “Only”
5. Then choose the class for restriction filler



With universal restriction



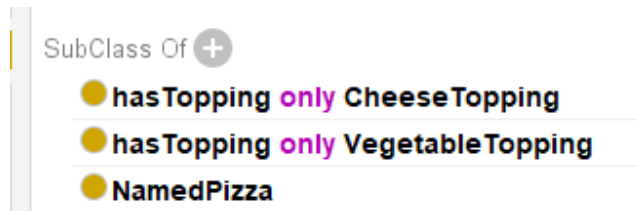
- Meaning:
 - if something is a member of the class VegetarianPizza it is necessary for it to be **a kind of Pizza** and it is necessary for it to **only have toppings that are kinds of CheeseTopping or kinds of VegetableTopping**
 - It also means the class VegetarianPizza that contains individuals that are **Pizzas and do not participate in any hasTopping relationships**
- If you want your pizza to **MUST** have and **ONLY** have CheeseTopping and VegetableTopping, then you should define it as ...



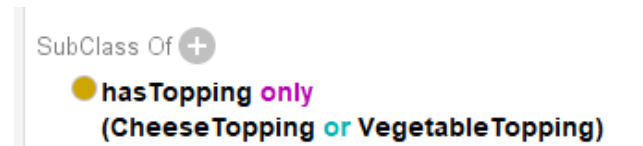


Remark on creating universal restriction

- If we create two universal restrictions --
 - one for CheeseTopping (***hasTopping only CheeseTopping***)
 - and one for VegetableTopping (***hasTopping only VegetableTopping***)



(a)



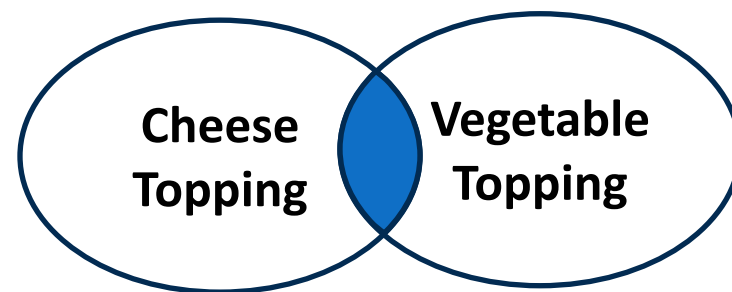
(b)

- When multiple restrictions are used (for any type of restriction) the total description is taken to be the intersection of the individual restrictions
- So, (a) would be equivalent to one restriction with a filler that is the intersection of *CheeseTopping* and *VegetableTopping*



SubClass Of +

- hasTopping **only** CheeseTopping
- hasTopping **only** VegetableTopping
- NamedPizza

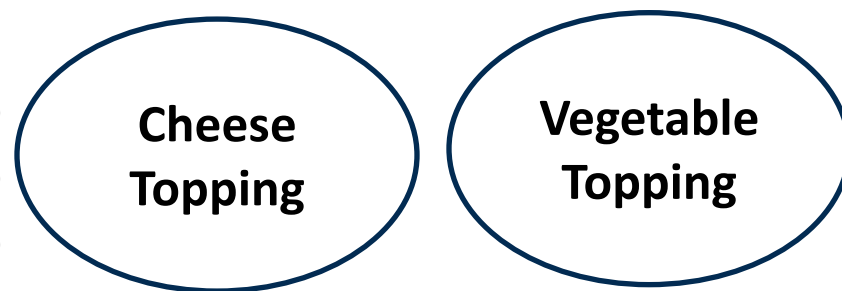


Instances of this class can have a hasTopping relationship with instances that belong to both the CheeseTopping and VegetableTopping classes.

-
- What if we declare CheeseTopping and VegetableTopping disjoint?

SubClass Of +

- hasTopping **only** CheeseTopping
- hasTopping **only** VegetableTopping
- NamedPizza



Instances of this class can not have hasTopping relationship!!



Cardinality Restriction

- Cardinality restriction
 - In OWL we can describe the class of individuals that have at least, at most or exactly a specified number of relationships with other individuals or datatype values
- For a given property ***P***
 - *Minimum Cardinality Restriction* specifies the minimum number of ***P*** relationships that an individual must participate in
 - *Maximum Cardinality Restriction* specifies the maximum number of ***P*** relationships that an individual can participate in.
 - *Exact Cardinality Restriction* specifies the exact number of ***P*** relationships that an individual must participate in

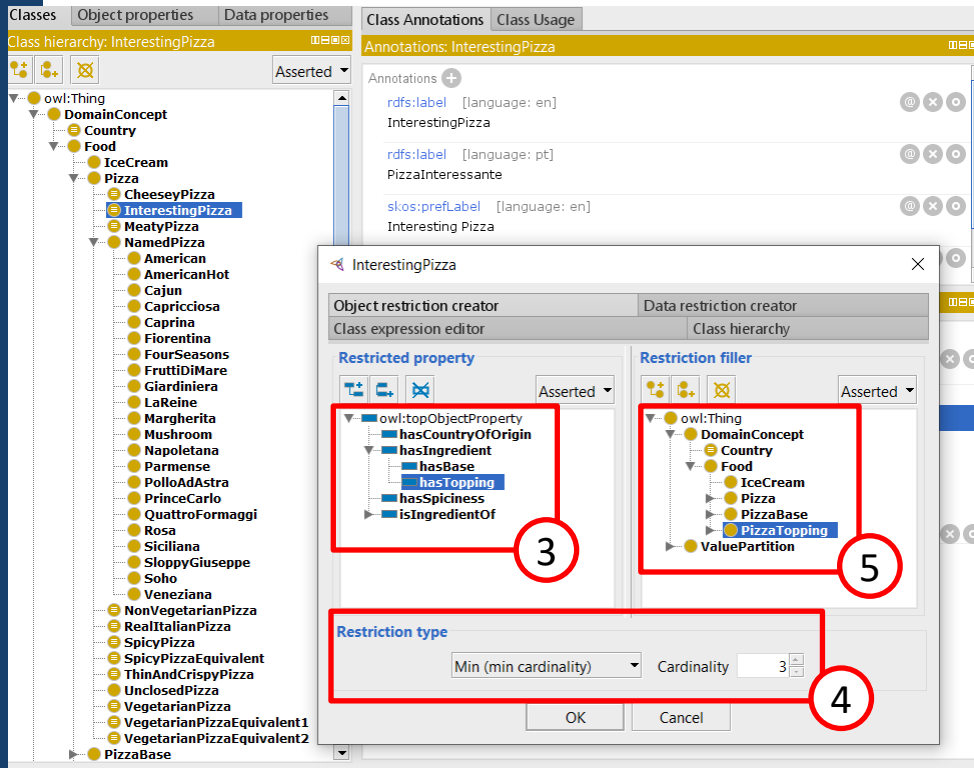


Setting Cardinality Restriction

Choose class: InterestingPizza

On description pane

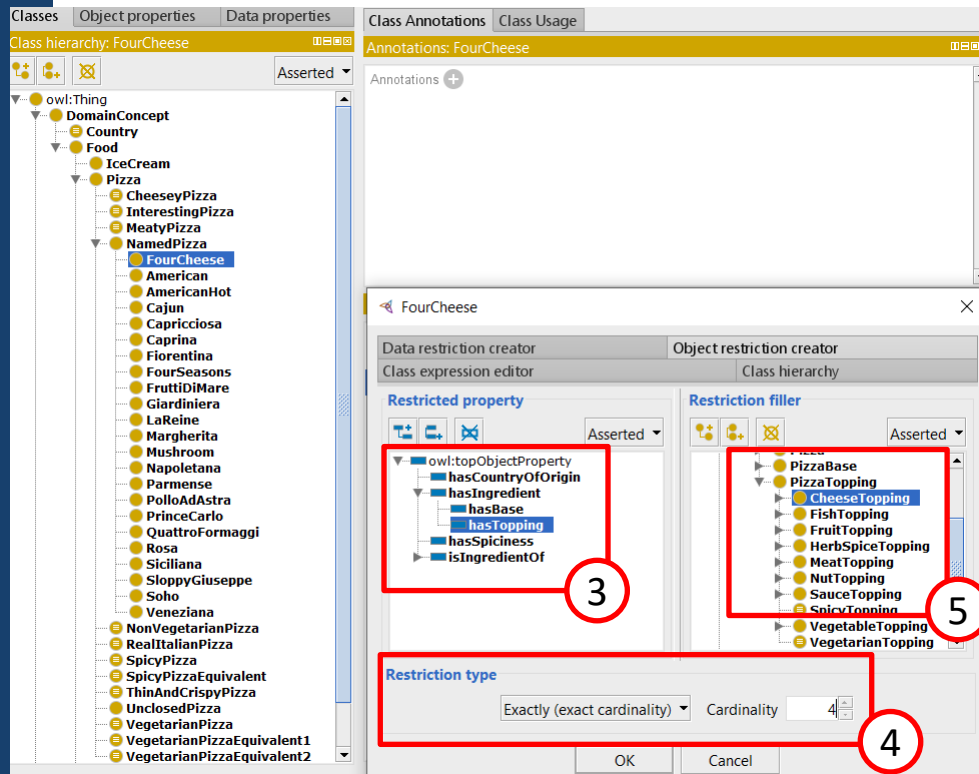
1. Click “add” (+) next to Subclass Of
2. Click tab “Object Restriction Creator”
3. Choose the desire property
 - Here is “hasTopping”
4. Choose “Restriction Type” to “Min” and set the value of minimum number
5. Choose the class(es) for property range



Choose **Max** for Maximum Cardinality Restriction



Setting Cardinality Restriction



First, create FourCheese class under NamedPizza

Then, on description pane

1. Click “add” (+) behind Subclass Of
2. Click tab “Object Restriction Creator”
3. Choose the desire property
 - Here is “hasTopping”
4. Choose “Restriction Type” to “Exact” and set the value of cardinality
5. Choose the class(es) for property range

- After creating this restriction
 - *FourCheesePizza* describes the set of individuals that are members of the class *NamedPizza* and that have exactly four *hasTopping* relationships with individuals of the *CheeseTopping* class
 - If we want it to have only four cheese toppings and no other toppings ...
 - we must add the keyword 'only' (the universal quantifier)

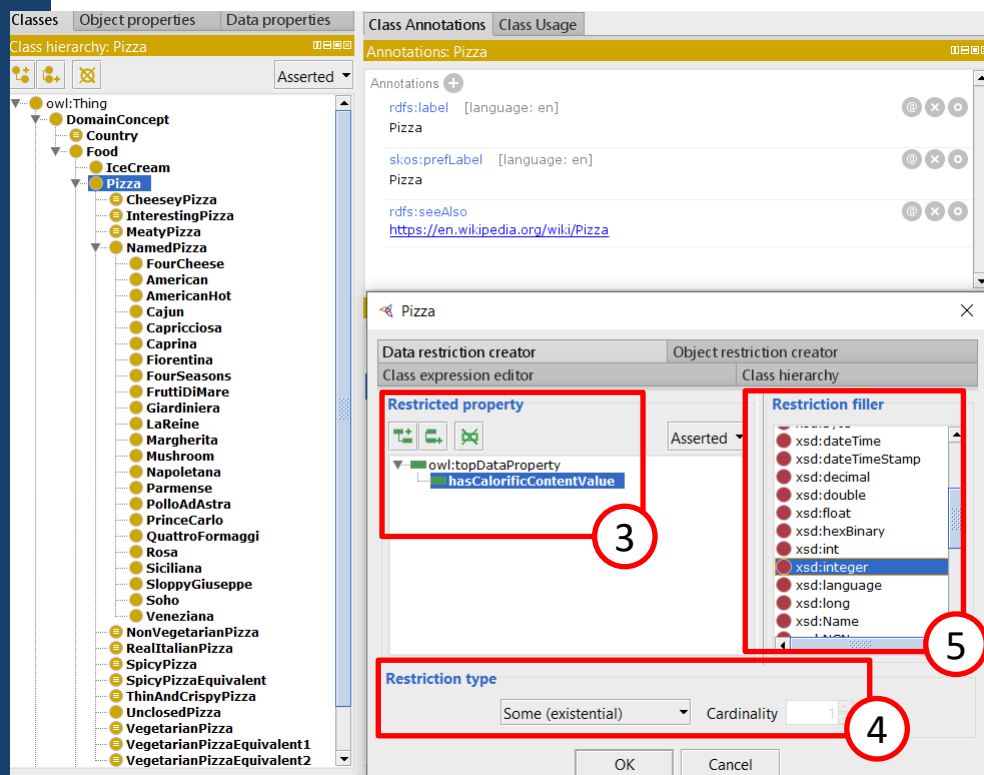


Datatype Property

- Datatype properties link an individual to an XML Schema Datatype value or rdf literal
 - They describe relationships between an individual and data values
- Datatype properties can be created using the “Datatype Properties view”
- We can also use the functional characteristic on data properties
 - By making datatype property ***D*** functional, we are saying that any individual with that property can only ever have one value of property ***D***

Restriction with Datatype Property

- A datatype property can also be used in a restriction
- For example: Create a datatype restriction to state that all Pizzas have a calorie value
 - All pizzas have at least one calorie value (and it must be integer)



Then, on description pane

1. Click “add” (+) behind Subclass Of
2. Click on tab “Data Restriction Creator”
3. Select the Datatype Property
 - Here is “**hasCalorificContentValue**”
4. Choose “Restriction Type” to “Some”
 - Here is “Integer”
5. Choose data type



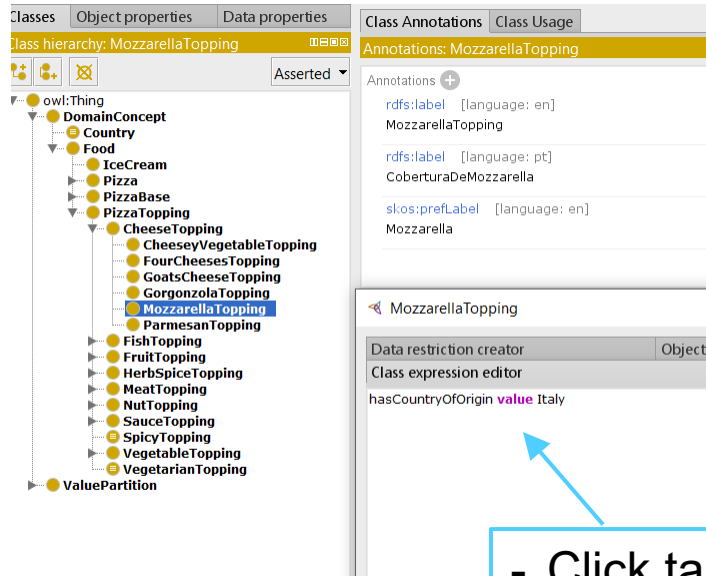
hasValue Restriction

- A **hasValue** restriction, denoted by the symbol $\exists$, describes the set of individuals that have at least one relationship along a specified property to a **specific individual**.
 - For example, the **hasValue restriction** *hasCountryOfOrigin* $\exists$ *Italy* (where *Italy* is an individual)
 - Describes the set of individuals that have at least one relationship along the **hasCountryOfOrigin** property to the specific individual *Italy*



Create hasValue Restriction

- For example: We want to state that mozzarella cheese (MozzarellaTopping) is from *Italy*
 - In pizza ontology, we have *Italy* as **individual** of class “**Country**”
- We can use a hasValue restriction



1. Make sure you have object property “*hasCountryOfOrigin*”
2. Choose the class
 - Here is “*MozzarellaTopping*”
3. On Description Pane, click “add” (+) behind “**Subclass Of**”

- Click tab “Class Expression Editor”
- Type expression -- *hasCountryOfOrigin value Italy*

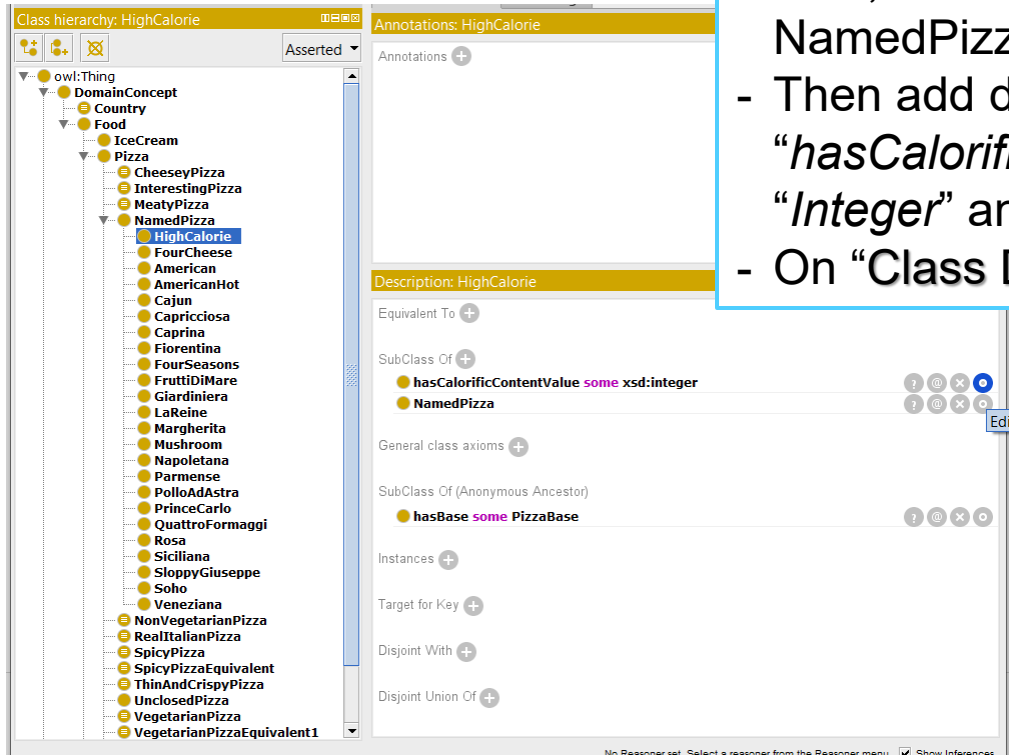


Setting possible values for datatype property

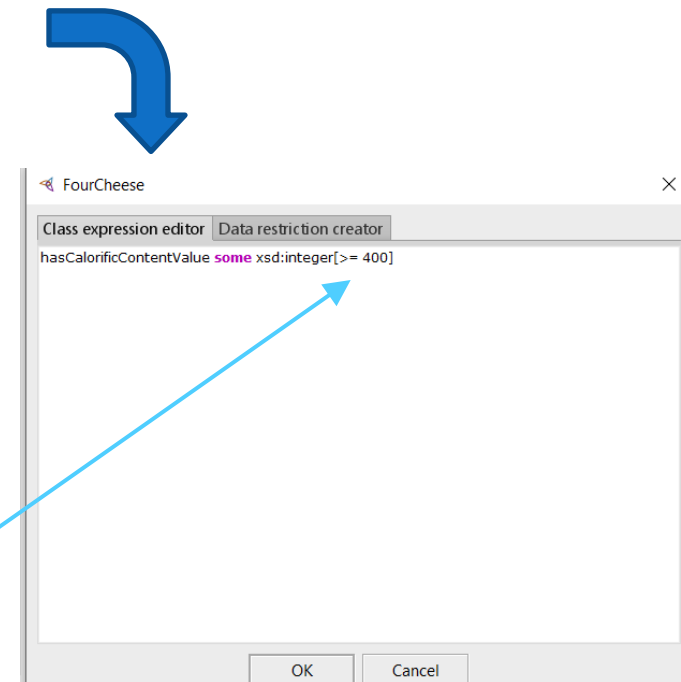
- We can further specialize the use of a datatype by specifying restrictions on the possible values
- For example: Define class HighCaloriePizza to be any pizza that has a calorie value equal to or higher than 400

Setting possible values for datatype property

- First, create class HighCalorie as subclass of NamedPizza
- Then add datatype property *hasCalorificContentValue* with data type *Integer* and restriction *Some*
- On "Class Description" pane, click "Edit" icon

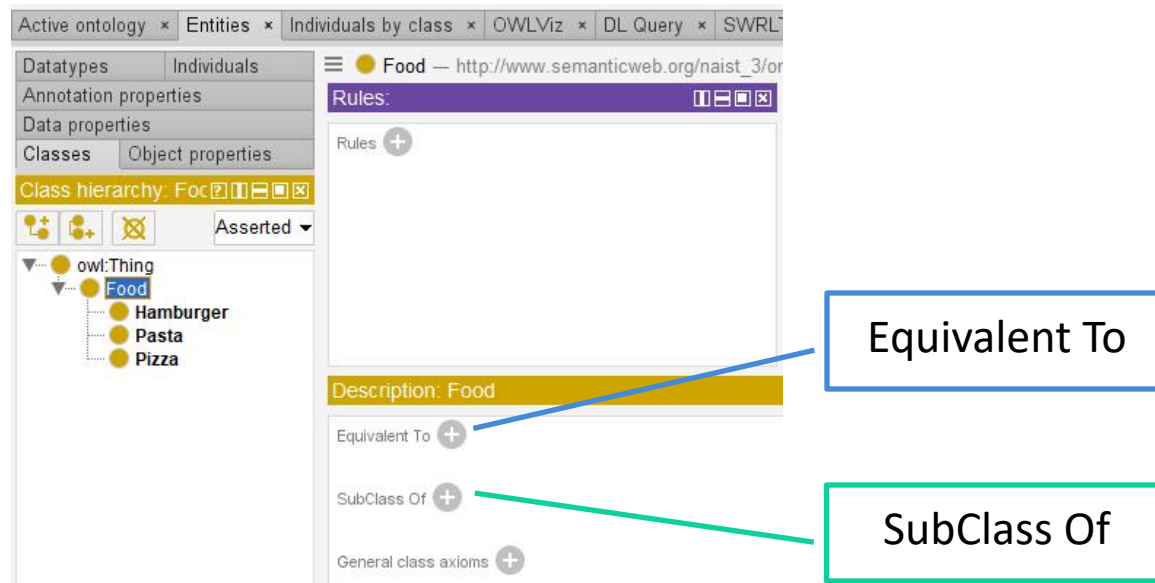


- Click tab "Class Expression Editor"
- Type expression $[>= 400]$ behind integer



Setting class restriction

- Two ways to set class restriction in Protégé.

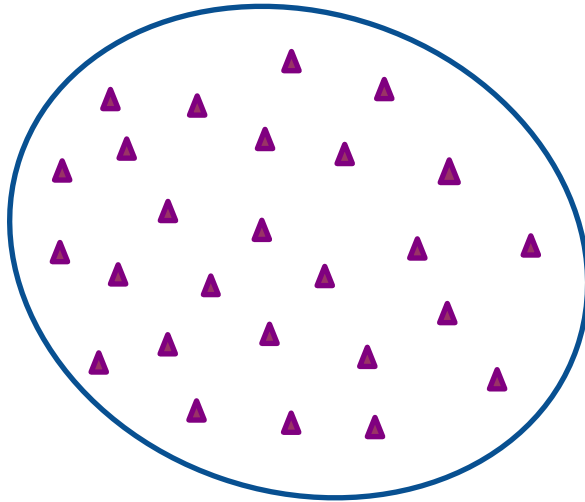


The screenshot shows the Protégé interface with the 'Food' class selected in the class hierarchy. The 'Food' class is a subclass of 'owl:Thing'. The 'Food' class has three subclasses: 'Hamburger', 'Pasta', and 'Pizza'. The 'Food' class is highlighted in blue. The 'Description: Food' tab is active, showing the 'Equivalent To' and 'SubClass Of' restrictions. The 'Equivalent To' restriction is highlighted with a blue box and labeled 'Equivalent To'. The 'SubClass Of' restriction is highlighted with a green box and labeled 'SubClass Of'.

- What is the difference?

Setting class restriction

Anonymous class



- When we create class restrictions, we are creating an “anonymous class”.

Class restrictions:

```
hasBase exactly 1 PizzaBase  
hasTopping some PizzaTopping
```

Set of instances that have exactly 1 pizza base and some pizza toppings

Anonymous class

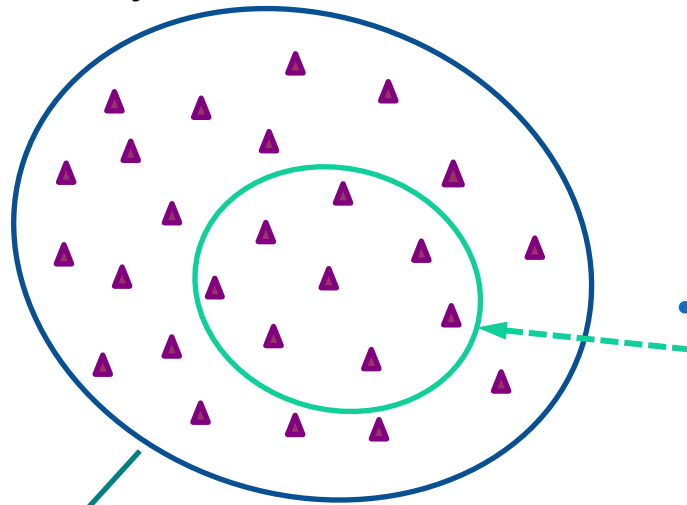
- A restriction is used to describe an anonymous class (an unnamed class).
- The anonymous class contains all of the individuals that satisfy the restriction – i.e. all of the individuals that have the relationships required to be a member of the class.



Setting class restriction: SubClass Of

- When we create class restrictions, we are creating an “anonymous class”.

Anonymous class



Set of instances that have exactly 1 pizza base and some pizza toppings

Class restriction:

```
hasBase exactly 1 PizzaBase  
hasTopping some PizzaTopping
```

- Suppose we set this class restrictions of Class *Pizza* at “Subclass Of”
 - It means we are saying that class *Pizza* is a subclass of this anonymous class
 - What does it mean?
 - Anything that is a member of *Pizza* class must follow this restriction.
 - However, we can’t say that anything that follows this restriction is a member of *Pizza* class.
 - We are creating “Necessary Condition”



Description: Margherita

Equivalent To

SubClass Of

- hasTopping** **only**
(MozzarellaTopping **or** TomatoTopping)
- hasTopping** **some** MozzarellaTopping
- hasTopping** **some** TomatoTopping
- NamedPizza**

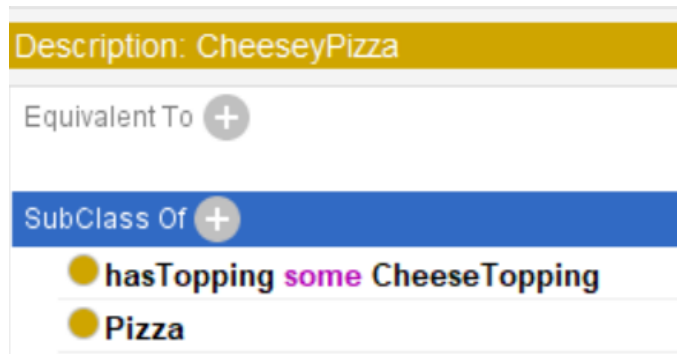
- From the above restriction:
 - We could say that MargheritaPizza is a subclass of Pizza and also a subclass of the things that have at least one topping that is MozzarellaTopping, one topping that is TomatoTopping, and nothing else.
 - This means, if something is MargheritaPizza, then it is a pizza and it is necessary for it to have at least one topping that is MozzarellaTopping, one topping that is TomatoTopping, and nothing else.

Necessary conditions for individual to become a member of that class



Necessary Conditions

- Necessary conditions can be read as, “If something is a member of this class then it is necessary to fulfill these conditions”.
- Suppose we defined “CheesyPizza” with this definition:



- if something is a `CheesyPizza`, then it is necessarily a `Pizza` and it is necessary for it to have at least one topping that is a kind of `CheeseTopping`

- But is every pizza with at least one `CheeseTopping` a `CheesyPizza`?

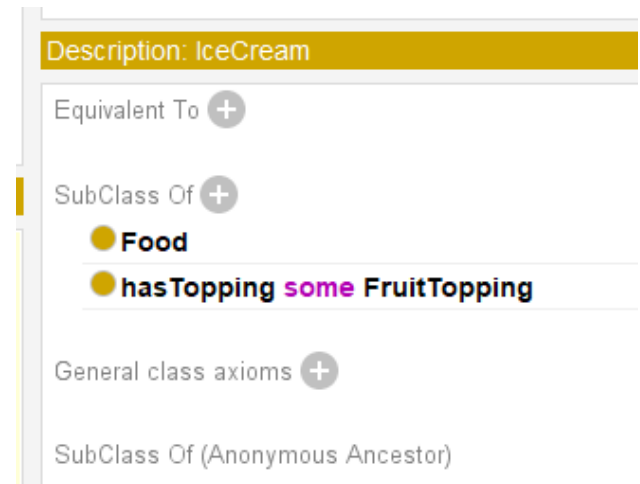
With necessary conditions alone, we cannot say that “If something fulfills these conditions then it must be a member of this class”.



Let's try

- From Description pane of IceCream class
- Click ⊕ behind 'Subclass of'
 - Add `hasTopping some FruitTopping`
- Then start reasoner

What will happen if we start reasoner?

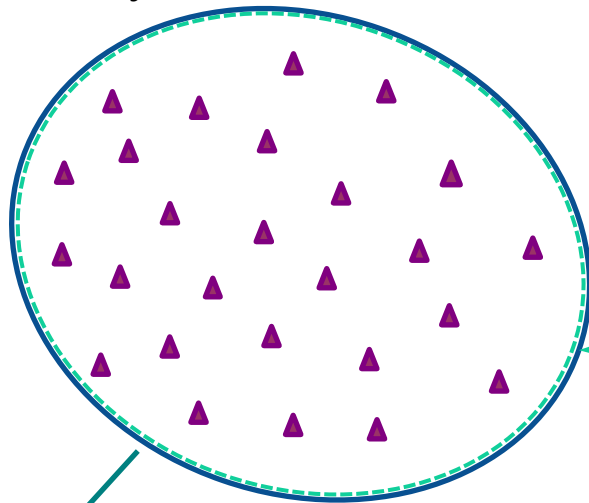


- Let's add back Pizza to the disjoint class of IceCream
- Then start reasoner

Setting class restriction: Equivalent To

- When we create class restrictions, we are creating an “anonymous class”.

Anonymous class



Set of instances that have exactly 1 pizza base and some pizza toppings

Class restrictions:

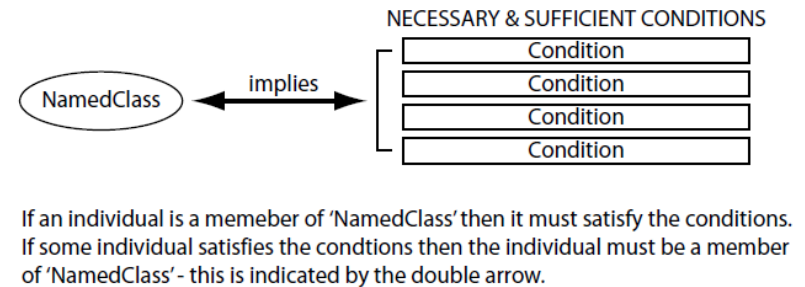
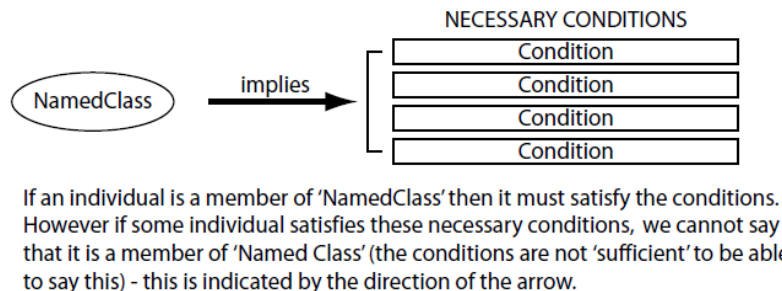
```
hasBase exactly 1 PizzaBase  
hasTopping some PizzaTopping
```

- Suppose we set this class restrictions of *Class Pizza* at “Equivalent To”
 - It means we are saying that class *Pizza* is equivalent to this anonymous class
 - What does it mean?
 - Anything that is a member of *Pizza* class must follow this restriction.
 - And anything that follows this restriction is a member of *Pizza* class.
- We are creating “Necessary and Sufficient Condition”



Necessary and Sufficient Conditions

- We need to change the conditions for a class from necessary conditions to *necessary AND sufficient* conditions.
 - This means that not only are the conditions necessary for membership of the class **A**, they are also sufficient to determine that any individual that satisfies them must be a member of the class **A**



A class that has at least one set of necessary and sufficient conditions is known as a **Defined Class**.

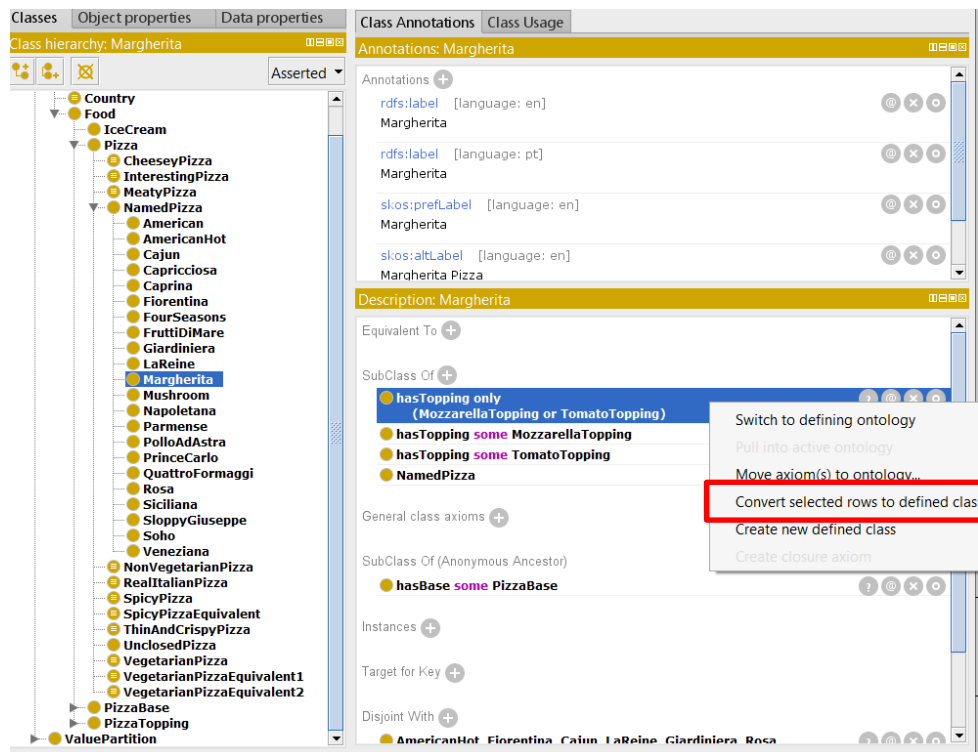


Necessary and Sufficient Conditions

- Necessary Condition
 - If class A is described using necessary conditions, then we can say that if an individual is a member of class A it must satisfy the conditions
 - BUT we cannot say that any (random) individual that satisfies these conditions must be a member of class A
- Necessary and Sufficient Condition
 - if class A is now defined using necessary and sufficient conditions, we can say that if an individual is a member of the class A it must satisfy the conditions
 - AND we can say that if any individual satisfies these conditions, then it must be a member of class A

Necessary and Sufficient Conditions

- To convert *Necessary* conditions to *necessary and sufficient* conditions, the conditions must be moved to under “Equivalent classes” header.
 - This can be done with the “Convert to defined class”

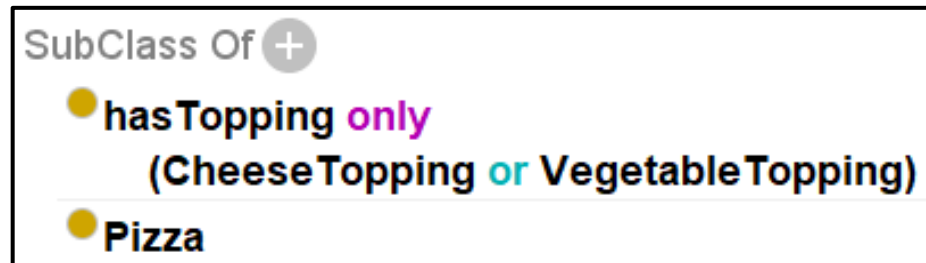


The screenshot displays the Protégé ontology editor with the 'Classes' tab selected. On the left, a class hierarchy for 'Margherita' is shown, including subclasses like 'American', 'Cajun', 'Capricciosa', etc. The main panel shows the 'Class Annotations' for 'Margherita', listing various annotations such as 'rdfs:label' and 'skos:prefLabel'. The 'Description' tab is also visible, showing the class's definition. A right-click context menu is open over the 'hasTopping only (MozzarellaTopping or TomatoTopping)' row, with the option 'Convert selected rows to defined class' highlighted by a red rectangle.

1. Select the rows that you want to convert,
2. Right click on the selected row and choose “convert selected rows to defined class”



Let's look at VegetarianPizza ...



- We know that any individual that satisfies these conditions must be a *VegetarianPizza*
- We can therefore convert the necessary conditions for *VegetarianPizza* into necessary and sufficient conditions
- This will also enable us to use the reasoner to determine the subclasses of VegetarianPizza



By converting the restrictions for VegetarianPizza to necessary and sufficient condition ...

SubClass Of +

- hasTopping **only**
(CheeseTopping **or** VegetableTopping)
- Pizza

Equivalent To +

- Pizza
and (hasTopping **only**
(CheeseTopping **or** VegetableTopping))

Necessary condition

- If x is a Vegetarian Pizza, then it can have only cheese or vegetable or both as its toppings.

Necessary and sufficient condition

- If x is a Vegetarian Pizza, then it can have only cheese or vegetable or both as its toppings.
- And if there is an instance of pizza that has only cheese or vegetable or both as its toppings, then it will be considered as Vegetarian Pizza.



Usefulness of Necessary and Sufficient Conditions

- Suppose we have class **B**
 - We know that any individuals that are members of class **B** also satisfy the conditions that define class **A**
- From the Necessary and Sufficient conditions
 - We can determine that class **B** is subsumed by class **A**
 - In other words, **B** is a subclass of **A**
- *Checking for class subsumption is a key task of a description logic reasoner*



Exercise: Determine subsumption relationship

Pizza1

Equivalent To:

hasTopping only (VegetableTopping or CheeseTopping)

hasTopping some VegetableTopping

hasTopping some CheeseTopping

Pizza2

Equivalent To:

hasTopping some CheeseTopping

Pizza3

Equivalent To:

hasTopping some VegetableTopping

hasTopping some MeatTopping

Pizza4

Equivalent To:

hasTopping some VegetableTopping



Primitive class VS Defined class

- **Primitive class**
 - Classes that **do not have any sets of necessary and sufficient conditions** (only have necessary conditions) are known as primitive classes.
- **Defined class**
 - Classes that have **at least one set of necessary and sufficient conditions** are known as defined classes
 - They have a definition, and any individual that satisfies the definition will belong to the class.



Exercise

- Add the following pizza menus to your pizza ontology with restrictions that will appropriately represent the characteristics of each pizza
- You may need to add other toppings to your ontology
- First: Create “MyPizza” class under “Pizza”
- Then: add the following pizzas with restrictions under “MyPizza”



Exercise (cont.)

- Pizza menus:
 - **Hawaiian Pizza** is a pizza that contains at least pineapple, ham, tomato sauce, and cheese as its toppings.
 - TomatoSauceTopping is a subclass of SauceTopping
 - Pineapple is a subclass of FruitTopping
 - **Spinach Deluxe Pizza** is a pizza that must have only spinach and cheese as its topping.
 - **Spicy Meat Lover Pizza** is a pizza that contain at least 4 types of meat, where one of them must be ham. Spicy level of this pizza is Hot.
 - **Pesto Chicken Pizza** is a pizza that must contain chicken and basil in its topping. Spicy level of this pizza is Medium.
 - Basil is a subclass of HerbSpiceTopping
 - **Super Cheese Pizza** is a pizza that contain only 5 kinds of cheeses as its topping.



Protégé for collaboration

- Collaborative Protégé
- Web protégé
 - <https://webprotege.stanford.edu>